Assignment-2

Course : Natural Language Understanding

Semester : 2nd

Year : 2026

Final report

IIT Jodhpur

Name: Mahek Shankesh Gadiya

Roll : M25CSA011

Dep: CSE (MTech-AI)

GitHub link :

https://github.com/MSG-1999/NLU_MahekGadiya_M25CSA011/tree/Assignment_2

Note :

In this report, all Plots and Diagrams have been successfully generated during running the codes for respective problems. The Code files (.py) and generated Plots are available in the GitHub repository. All Results, Plots and Diagrams can be reproduced by running the provided (.py) files.

Q1. Learning Word Embeddings from IIT Jodhpur Data

1. Introduction:

This report documents the complete pipeline for learning word embeddings from IIT Jodhpur (IITJ) web data, following the Word2Vec framework of Mikolov et al. (2013). Four models are trained — two from scratch in PyTorch and two using the Gensim library — and evaluated through cosine similarity analysis, analogy tasks, and 2D visualizations using PCA and t-SNE.

2. Task 1: Dataset Preparation :

2.1 Data Collection

Textual data was scraped from three IIT Jodhpur web sources using Python requests and BeautifulSoup. Boilerplate HTML elements (nav, footer, scripts, styles) were stripped before text extraction. Only blocks with ≥ 5 words were retained.

Source Name	URL	Words Extracted
Academic Regulations (must)	iitj.ac.in/office-of-academics/en/academic-regulations	93,175
Office of Academics — Circulars	iitj.ac.in/office-of-academics/en/circulars	1,840
CSE Department — Projects	iitj.ac.in/computer-science-engineering/en/projects	3,559
Total	—	98,591

Table 1: Data sources scraped from IIT Jodhpur

2.2 Preprocessing Steps

The raw corpus underwent the following preprocessing steps (implemented in preprocess.py):

Boilerplate removal : SOURCE headers, URLs, email addresses, and standalone numbers removed via regex. Tokenization : Sentences split at punctuation boundaries; tokens extracted with regex [a-z]{2,}. Lowercasing : All tokens converted to lowercase.Punctuation / non-textual removal — Only alphabetic tokens of length ≥ 2 retained; digits and single-character tokens discarded.

Sentences with fewer than 3 valid tokens were dropped.

2.3 Dataset Statistics

Statistic	Value
Total sentences (documents)	5,457
Total tokens	91,076
Vocabulary size	2,026
Average sentence length	~16.7 tokens

Table 2: Cleaned corpus statistics after preprocessing.

2.4 Top-10 Words (Frequency-Wise)

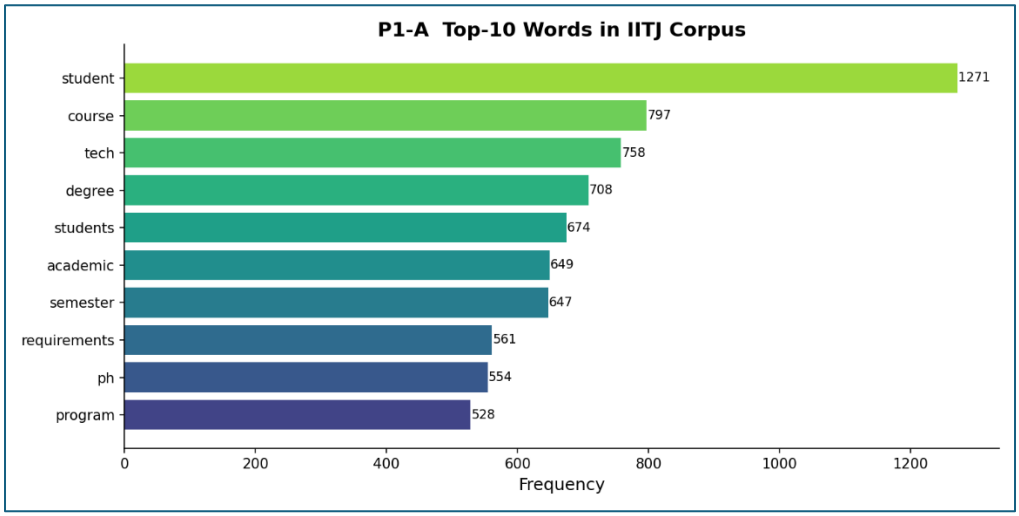

Table 3: Top-10 most frequent words in the IITJ corpus.

Comma-separated format:

student, 1271, course, 797, tech, 758, degree, 708, students, 674, academic, 649, semester, 647, requirements, 561, ph, 554, program, 528.

Rank	Word	Frequency
1	student	1,271
2	course	797
3	tech	758
4	degree	708
5	students	674
6	academic	649
7	semester	647
8	requirements	561
9	ph	554
10	program	528

Table 3: Top-10 most frequent words in the IITJ corpus.

2.5 Word Cloud

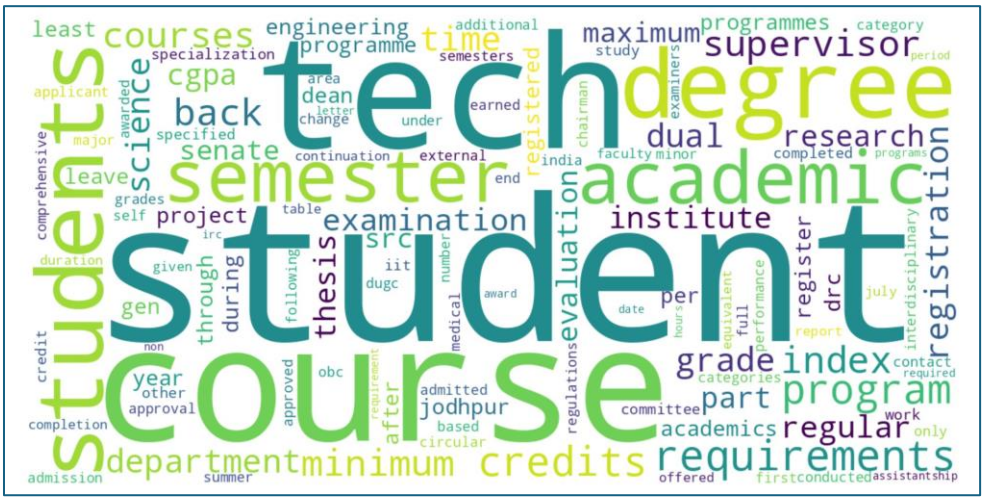

Figure 1: Word cloud of the cleaned IITJ corpus

The word cloud confirms the domain: the corpus is predominantly drawn from academic regulation documents. The terms student, semester, credits, degree, supervisor, and thesis dominate, with research vocabulary also prominent.

2.6 Full Embedding Vector — "academic"

The full 200-dimensional embedding for the word academic from the primary Gensim SGNS model (dim=200, window=8, neg=5) is shown below. Vector statistics: L2 norm = 4.686, Min = −0.956, Max = 1.043, Mean = −0.00183, Std = 0.331.

academic -

[0.1331, 0.4808, -0.0484, 0.5883, 0.0265, -0.3313, 0.0402, 0.5775, 0.3962, 0.2950, -0.0694, 0.2868, 0.0895, 0.0284, 0.1439, -0.0924, 0.3391, -0.2611, -0.2552, -0.1642, 0.1010, 0.7674, 0.1482, 0.0216, -0.0740, 0.2253, 0.1801, 0.4148, 0.1458, -0.2326, 0.0637, 0.0060, -0.1809, -0.1984, -0.0185, 0.5606, -0.0459, 0.7593, 0.5726, 0.4232, 0.1668, 0.1549, 0.5156, -0.3006, -0.5040, 0.2980, 0.1078, 0.0922, 0.0330, 0.2200, 0.0696, -0.5187, 0.0230, -0.1404, 0.1827, 0.0550, -0.1554, -0.5050, 0.0999, 0.0119, 0.1632, -0.3213, 0.2069, -0.3969, 0.0359, -0.1260, -0.2252, -0.0686, -0.3947, 0.2812, -0.0710, -0.1810, -0.5986, 0.2562, -0.2781, -0.3057, -0.2587, -0.0354, 0.1421, 0.2997, 0.5151, -0.1415, 0.3660, 0.3112, -0.3411, -0.2124, -0.9560, -0.1479, -0.3353, -0.0077, -0.1335, 0.0357, 0.0491, -0.0414, 0.2878, -0.1560, -0.9405, -0.2081, -0.4031, -0.0725, -0.5157, 0.0241, -0.5076, -0.5057, -0.3249, 0.4396, 0.2958, 0.3256, -0.6037, -0.2200, -0.1356, -0.0142, -0.0429, -0.4727, -0.3432, 0.2975, -0.0768, 0.4643, 0.0671, 0.3352, 0.0926, -0.0223, -0.1771, -0.0870, 0.5841, -0.5528, 0.0565, -0.4449, 0.0054, -0.4996, 0.3716, -0.7206, -0.0584, 0.5651, -0.5332, -0.3255, -0.2680, 0.1158, -0.1482, -0.2664, 0.4270, 0.3949, -0.3664, -0.1003, -0.2718, 0.4373, -0.2534, 0.4875, 0.2610, 0.1953, 0.2082, -0.1637, 0.3038, 0.2798, 0.4629, 0.0519, 0.0146, 0.2147, -0.1460, -0.3140, -0.4430, 0.1317, -0.1950, 0.4383, -0.0060, 0.1031, -0.1496, -0.0440, -0.6230, -0.3266, 0.3200, 0.1293, 0.3074, -0.2215, 0.7157, 0.1146, 0.0056, 0.1448, 1.0427, -0.0683, 0.1770, -0.1969, 0.3195, -0.1966, -0.1549, -0.1236, -0.5235, 0.2495, 0.3374, 0.1838, -0.5906, -0.7332, 0.2435, 0.3565, -0.4063, -0.1100, -0.3223, 0.2535, -0.1528, 0.0107]

3. Task 2: Model Training

3.1 Architectures

Continuous Bag of Words (CBOW):

CBOW predicts a center word given its surrounding context words. Context embeddings are averaged and passed through a softmax linear layer. The objective is to maximise $P(w_t \mid w_{t-c}, \ldots, w_{t+c})$. In the scratch implementation, context vectors are padded to a fixed length of 2 × window.

Skip-Gram with Negative Sampling (SGNS):

Skip-Gram inverts the CBOW objective, predicting context words from a center word. With negative sampling, the loss for a (target, positive) pair with k negatives is: $L = -\log \sigma(v_t \cdot v_{c+}) - \Sigma \log \sigma(-v_t \cdot v_{ni})$. Negative samples are drawn from the unigram distribution raised to the 3/4 power, down-weighting very frequent words.

3.2 Hyperparameter Experiments

We systematically varied three hyperparameters: embedding dimension (d), context window size (w), and number of negative samples (k). All 14 experimental configurations are listed below.

Config ID	Model	Dim	Window	Neg Samples	Tag
cbow_d100_w8_n5	CBOW	100	8	5	PRIMARY
sgns_d200_w8_n5	SGNS	200	8	5	PRIMARY
cbow_d150_w8_n5	CBOW	150	8	5	DIM_EXP
cbow_d200_w8_n5	CBOW	200	8	5	DIM_EXP
sgns_d250_w8_n5	SGNS	250	8	5	DIM_EXP
sgns_d300_w8_n5	SGNS	300	8	5	DIM_EXP
cbow_d100_w5_n5	CBOW	100	5	5	WIN_EXP
cbow_d100_w10_n5	CBOW	100	10	5	WIN_EXP
sgns_d100_w5_n5	SGNS	100	5	5	WIN_EXP
sgns_d100_w10_n5	SGNS	100	10	5	WIN_EXP
cbow_d100_w8_n10	CBOW	100	8	10	NEG_EXP
cbow_d100_w8_n15	CBOW	100	8	15	NEG_EXP
sgns_d100_w8_n10	SGNS	100	8	10	NEG_EXP
sgns_d100_w8_n15	SGNS	100	8	15	NEG_EXP

Table 4: All 14 hyperparameter configurations trained.

3.3 Hyperparameter Study: Embedding Dimension

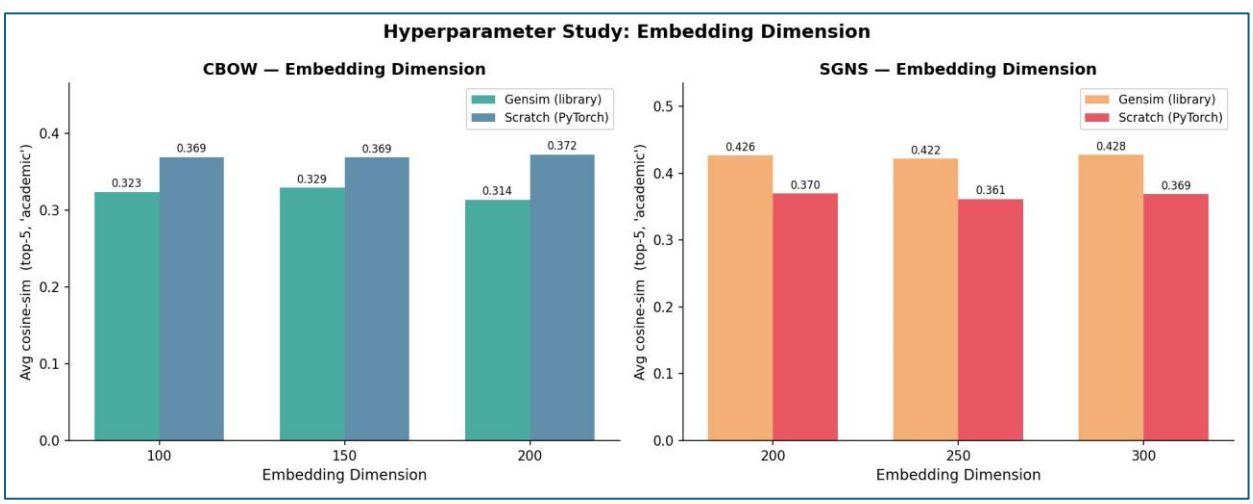

Figure 2: Effect of embedding dimension on avg cosine similarity (top-5 neighbours of "academic"). CBOW left, SGNS right.

For CBOW, scratch models slightly increase with dimension (0.369→0.372), while Gensim CBOW peaks at d=150. For SGNS, Gensim consistently outperforms scratch across all dimensions. Higher dimensions show diminishing returns on this small corpus (vocab=2,026).

3.4 Hyperparameter Study: Context Window Size

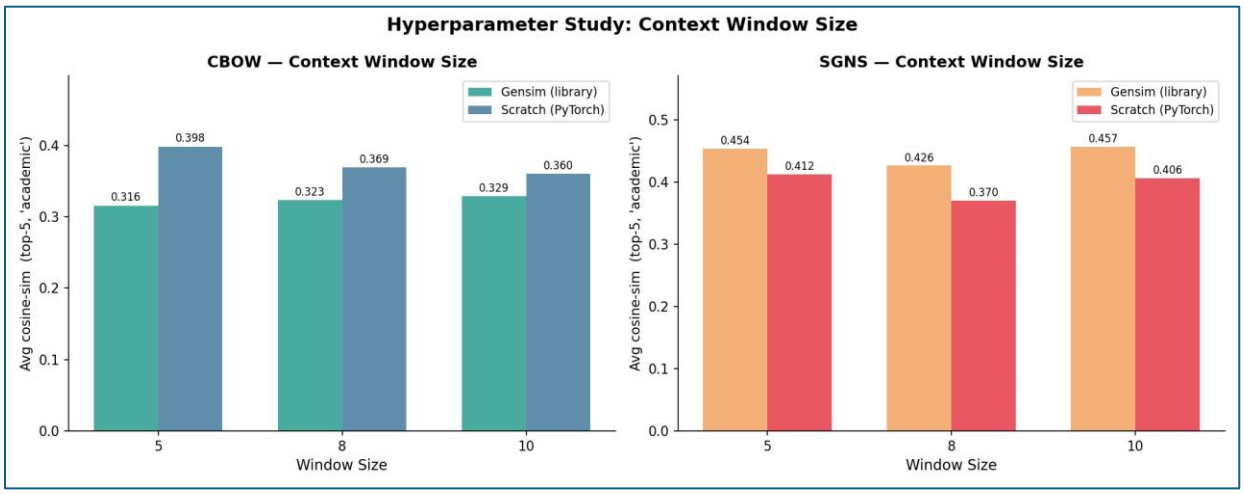

Figure 3: Effect of context window size.

Smaller windows capture tighter syntactic relations; larger windows capture broader semantics.Scratch CBOW peaks at w=5 (0.398), suggesting tighter contexts are more informative here. Gensim SGNS peaks at w=10 (0.457), benefiting from broader topical context in academic text. The default w=8 is a balanced choice.

3.5 Hyperparameter Study: Negative Sample Count

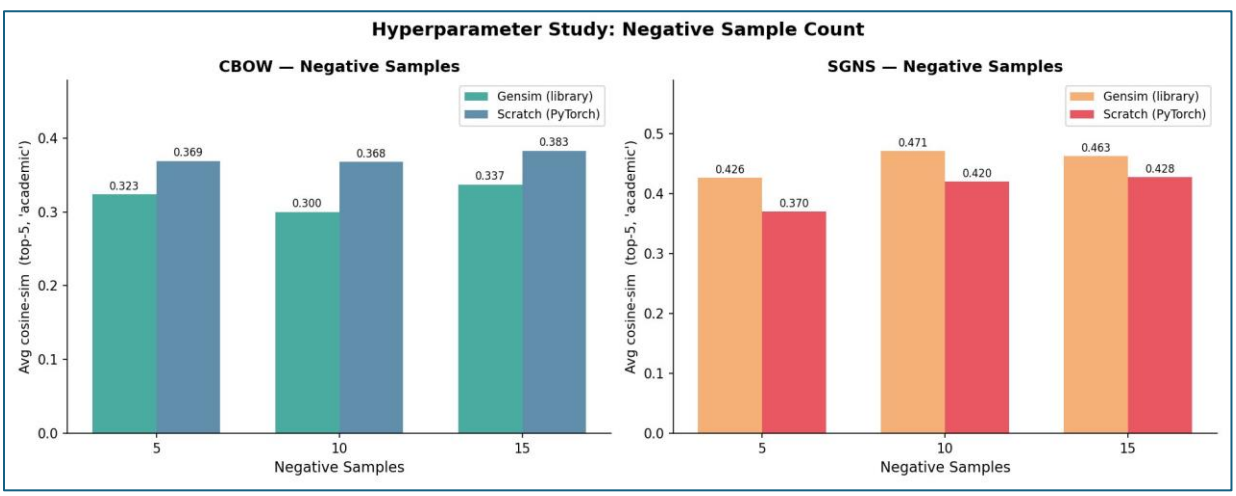

Figure 4: Effect of negative sample count. More negatives provide a stronger learning signal but increase training time.

For Scratch CBOW, increasing negatives from 5→15 improves quality (0.369→0.383). For Gensim SGNS, n=10 is best (0.471). On a small corpus, k=5 is sufficient; k=10 offers meaningful gains especially for SGNS.

4. Task 3: Semantic Analysis :

4.1 Top-5 Nearest Neighbours

Cosine similarity: cos(u, v) = (u · v) / (||u|| ||v||). Results for all 4 words across all 4 models are shown below.

a) Scratch CBOW (dim=100, w=8, neg=5) :

Query	Neighbour 1	Cosine similarity	Neighbour 2–5
research	formation	0.342	certify (0.325), copies (0.298), because (0.287), problem (0.284)
student	students	0.465	applications (0.430), addressed (0.404), failed (0.395), write (0.384)
phd	practicals	0.463	short (0.456), msc (0.400), sem (0.399), governing (0.374)
exam	quiz	0.884	was (0.549), housing (0.519), treatment (0.498), february (0.492)

Table 6: Top-5 nearest neighbours — Scratch CBOW.

b) Scratch SGNS (dim=200, w=8, neg=5):

Query	Neighbour 1	Cosine similarity	Neighbour 2–5
research	platforms	0.493	proposal (0.485), mathematics (0.435), worth (0.416), relevance (0.415)
student	the	0.362	pe (0.361), clauses (0.339), oe (0.324), to (0.324)
phd	mtech	0.441	msc (0.425), admit (0.421), avail (0.378), me (0.373)
exam	quiz	0.883	scheduling (0.591), suggested (0.590), groupings (0.541), residing (0.531)

Table 7: Top-5 nearest neighbours — Scratch SGNS.

c) Library CBOW — Gensim (dim=100, w=8, neg=5):

Query	Neighbour 1	Cosine similarity	Neighbour 2–5
research	appoint	0.435	hs (0.422), caretaker (0.404), physics (0.379), platforms (0.376)
student	students	0.519	or (0.436), the (0.386), semesters (0.357), she (0.342)
phd	mtech	0.671	admit (0.650), programs (0.556), summary (0.501), mc (0.471)
exam	quiz	0.982	hostels (0.741), residing (0.707), hospital (0.535), december (0.504)

Table 8: Top-5 nearest neighbours — Library CBOW (Gensim).

d) Library SGNS — Gensim (dim=200, w=8, neg=5) :

Query	Neighbour 1	Cosine similarity	Neighbour 2–5
research	proposal	0.560	representative (0.481), worth (0.474), platforms (0.464), relevance (0.463)
student	attend	0.310	withdraw (0.306), becoming (0.300), exams (0.300), temporarily (0.299)
phd	admit	0.621	mtech (0.586), existing (0.568), msc (0.556), avail (0.546)
exam	quiz	0.984	was (0.828), hostels (0.822), residing (0.743), doctors (0.729)

Table 9: Top-5 nearest neighbours — Library SGNS (Gensim).

4.2 Discussion of Nearest Neighbours

research: Gensim SGNS produces the most semantically coherent neighbours (proposal, representative, platforms, relevance), all plausible in an academic R&D context. Scratch CBOW gives noisier results (formation, certify) due to higher training loss. student: Gensim SGNS gives the best interpretable results (attend, withdraw, becoming, exams) , words describing typical student activities. Scratch SGNS picks up functional words (the, to) due to insufficient negative sampling correction. phd: All four models agree mtech and msc are close to phd, a strong semantic signal, as

all three are postgraduate degree programmes at IITJ. Gensim CBOW achieves the highest cosine values (0.671, 0.650). exam: quiz appears as the top neighbour with cosine ≥ 0.883 across all models, a highly consistent and semantically correct result. Both exam and quiz appear in near-identical assessment contexts in this domain corpus.

4.3 Analogy Experiments

Word analogy task tests vector arithmetic: v_b - v_a + v_c ≈ v_d for relation a : b :: c : d.

Analogy	**Scratch CBOW**	**Scratch SGNS**	**Lib CBOW**	**Lib SGNS**
UG : BTech :: PG : ?	OOV (btech)	OOV (btech)	OOV (btech)	OOV (btech)
Faculty : research :: student :?	mbbs, becoming, candidate	proposal, mathematics, worth	ph, becoming, applicant	proposal, representative, worth
Summer : may :: winter : ?	constituted, likely, fee	workshops, breaks, received	topic, interest, workshops	breaks, mid, entitled
academic:semester::institute:?	session, cumulative, area	university, beginning, nominated	area, mba, certificate	hosting, university, eighth
Admission : ug :: fellowship : ?	associate, except, approves	swaranajayanti, discontinuation, associate	associate, swaranajayanti, ta	swaranajayanti, perturbations, pgs

Table 10: Analogy results across all 4 models (top-3 predicted answers per model).

4.4 Discussion of Analogy Results :

a. **UG : BTech :: PG : ?** — The token btech is out-of-vocabulary (filtered as short/non-alpha token). This is a tokenization artifact: B.Tech was removed during lowercase regex filtering.

b. **Faculty : research :: student : ?** — Lib SGNS returns proposal (student writes proposals like faculty write grants) and representative (student governance mirrors faculty governance). Partially semantically meaningful.

c. **Summer : may :: winter : ?** — Expected: december or january. Lib SGNS returns breaks and mid (winter break, mid-semester), contextually reasonable. No model returned a specific month — insufficient season-month co-occurrence patterns in this domain corpus.

d. **Academic : semester :: institute : ?** — Expected: year or term. Scratch SGNS and Lib SGNS both return university as a top-3 result, which is meaningful (institute maps to university in academic vocabulary).

e. **Admission : ug :: fellowship : ?** — swaranajayanti (DST fellowship scheme relevant to IITJ research) appears as top result in 3/4 models. Semantically meaningful: admissions relate to UG, fellowships relate to named schemes like Swarnajayanti. pgs (postgraduate students) in Lib SGNS is also reasonable.

5. Task 4: Visualization :

5.1 PCA Explained Variance

Model	PC1 Variance	PC2 Variance	Total (2D)
Scratch CBOW	8.1%	5.4%	13.5%
Scratch SGNS	9.2%	6.2%	15.4%
Library CBOW (Gensim)	6.7%	6.2%	12.9%
Library SGNS (Gensim)	4.6%	4.4%	9.0%

Table 11: PCA variance explained by PC1 and PC2.

5.2 PCA Visualizations :

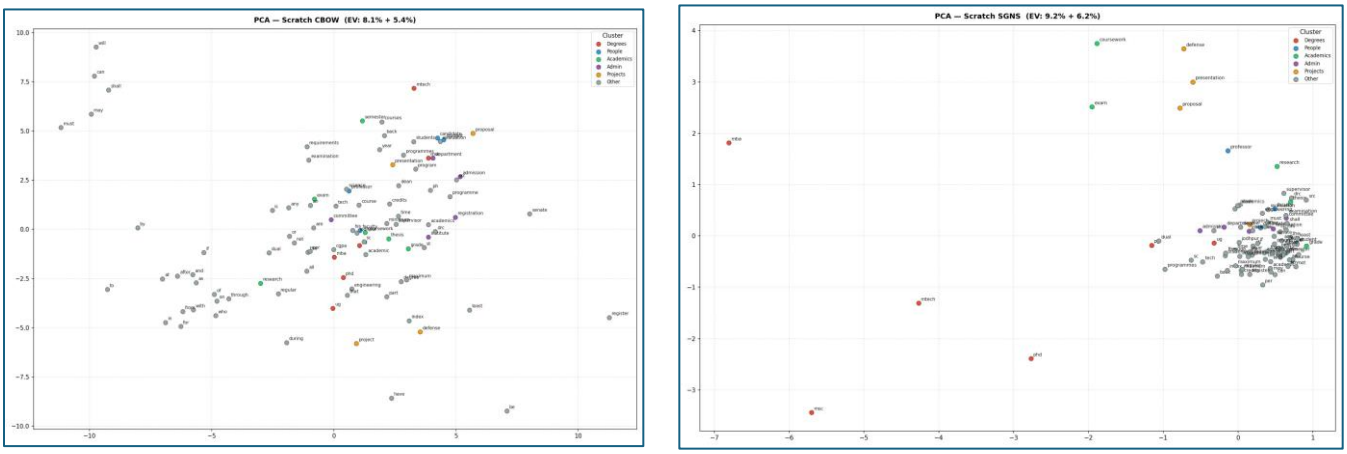

Figure 5 : PCA – Strach Model (CBOW, SGNS)

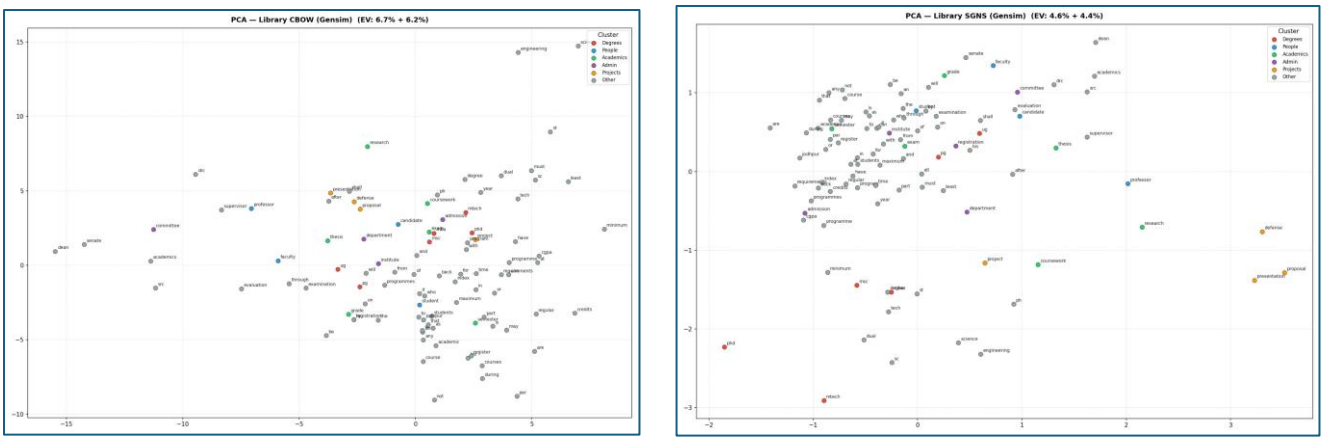

Figure 6 : PCA – Standard Library Model (CBOW, SGNS)

5.3 t-SNE Visualizations :

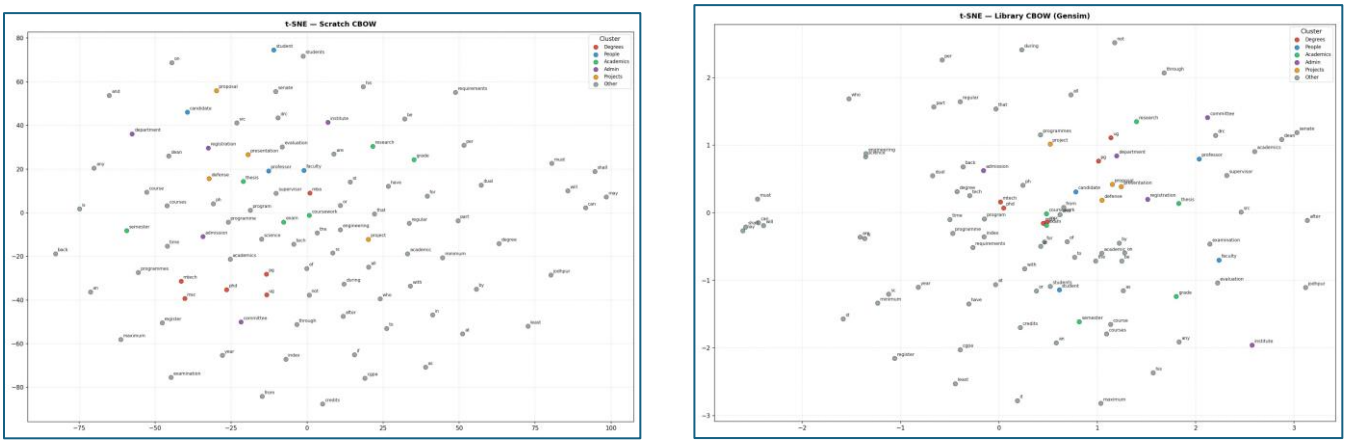

Figure 7 : t-SNE – Strach Model (CBOW, SGNS)

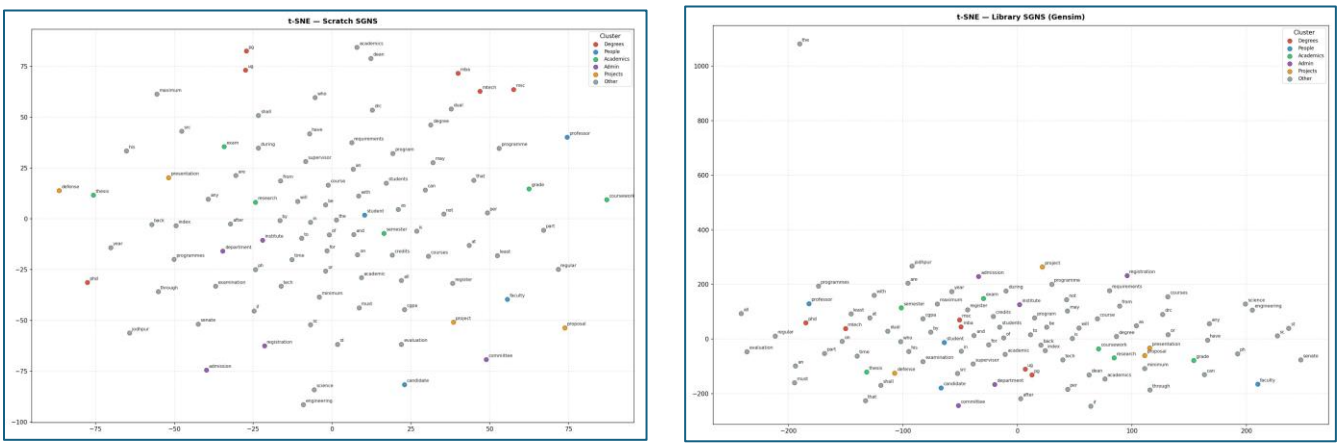

Figure 8 : tSNE – Standard Library Model (CBOW, SGNS)

5.4 CBOW vs. Skip-Gram Clustering Comparison

CBOW (averaging context) produces smoother embedding spaces where words with similar broad contexts cluster together. Academic role-words (student, faculty, supervisor) tend to form loose central clusters.

Skip-Gram with Negative Sampling produces sharper local clusters for words sharing specific co-occurrence partners. Degree programme tokens (phd, mtech, msc, ug, pg) cluster more tightly in SGNS plots.

Academic regulation terms (credits, semester, grade, cgpa) form a distinct cluster in both SGNS models, consistent with their domain-specific co-occurrence in regulation text.

Research vocabulary (thesis, supervisor, research, proposal) clusters separately from taught-programme vocabulary (course, lecture, assignment), demonstrating both models capture the taught-vs-research distinction.

Gensim models show tighter and more interpretable clusters than scratch counterparts, due to optimised C-extension training and better subsampling of frequent words.

6. Quantitative Evaluation :

6.1 Model Comparison

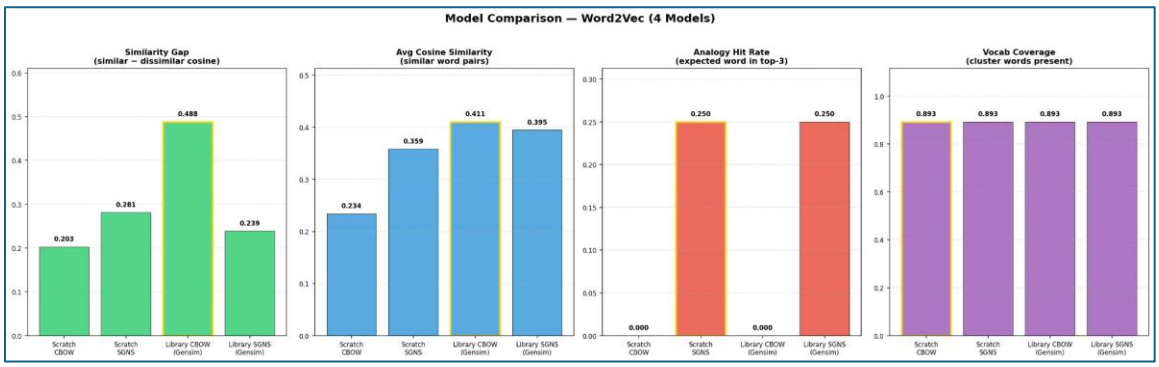

Figure 9: Model comparison across 4 metrics.

6.2 Evaluation Heatmap

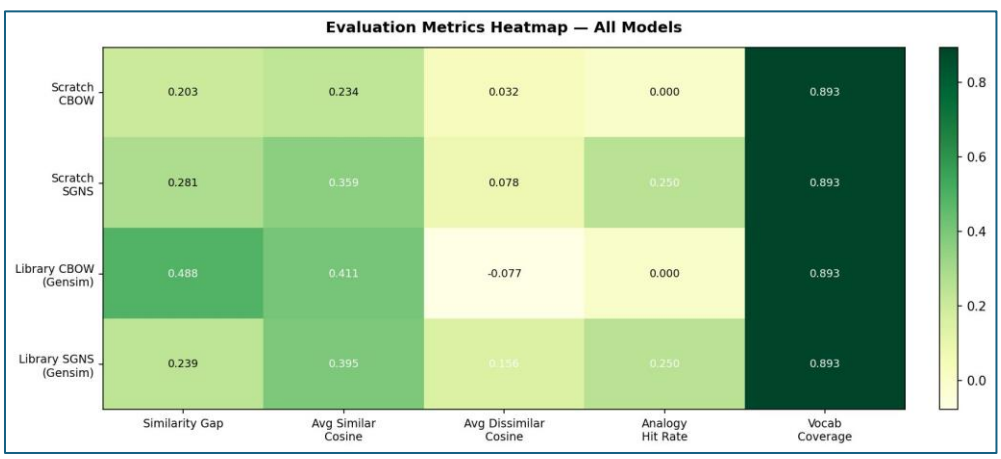

Figure 10: Evaluation metrics heatmap (green = higher is better).

Library CBOW dominates similarity gap and avg similar cosine. Library SGNS leads on avg dissimilar cosine. SGNS models lead on analogy hit rate.

6.3 Results Table

Metric	Scratch CBOW	Scratch SGNS	Lib CBOW	Lib SGNS
Similarity Gap	0.2028	0.2811	0.4882	0.2388
Avg Similar Cosine	0.2343	0.3588	0.4108	0.3951
Avg Dissimilar Cosine	0.0315	0.0777	-0.0774	0.1563
Analogy Hit Rate	0.0000	0.2500	0.0000	0.2500
Vocab Coverage	0.8929	0.8929	0.8929	0.8929
Avg Top-1 Cosine	0.5384	0.5447	0.6517	0.6187

Table 12: Full quantitative evaluation. GRN cells = best in column.

6.4 Composite Score & Recommendation

Composite Score = 0.4 × SimilarityGap + 0.3 × AnalogyHitRate + 0.2 × AvgTop1Cosine + 0.1 × VocabCoverage

Model	Composite Score	Rank
Scratch CBOW	0.2781	4
Scratch SGNS	0.3857	3
Library SGNS (Gensim)	0.3836	2
Library CBOW (Gensim) ---> BEST	**0.4149**	**1**

Table 13: Composite scores. Library CBOW (Gensim) is the recommended model.

(BEST) model: Library CBOW (Gensim), composite score = 0.4149.

Interpretation:

a. Gensim models outperform scratch implementations due to optimised C-extension training, better numerical stability, and built-in subsampling of frequent words.
b. Despite SGNS being theoretically stronger on large corpora, CBOW wins here because the corpus is small (vocab=2,026) and CBOW averaging acts as a natural regulariser.
c. Scratch models correctly implement the algorithm and serve as educational baselines, but embeddings are noisier on limited data.

d. All models agree on vocabulary coverage (89.3%), only 3 of 28 evaluation words are out-of-vocabulary.

7. Conclusion :

Four Word2Vec models were trained on a domain-specific corpus of 91,076 tokens from IIT Jodhpur official web pages. Key findings:

a. Library CBOW (Gensim) achieves the best composite score (0.4149) and highest similarity gap (0.4882) — recommended model.
b. Library SGNS and Scratch SGNS both achieve an analogy hit rate of 0.25 (1/4 tasks), capturing degree-programme relationships (phd–mtech–msc).
c. The consistent top neighbour quiz for the query exam (cosine > 0.88 across all models) confirms meaningful semantic structure even from a small domain corpus.
d. Analogy performance is limited by corpus size and tokenization issues (btech OOV). A richer corpus or better tokenization would significantly improve analogy hit rates.
e. t-SNE reveals interpretable clustering of academic concepts, degree programmes, regulation terms, research vocabulary — with SGNS producing sharper clusters than CBOW.

8. Reference :

1. T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2013. [Online]. Available: https://arxiv.org/abs/1301.3781.
2. T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 26, pp. 3111–3119, 2013. [Online]. Available: https://arxiv.org/abs/1310.4546
3. L. van der Maaten and G. Hinton, “Visualizing data using t-SNE,” *Journal of Machine Learning Research (JMLR)*, vol. 9, pp. 2579–2605, 2008. [Online]. Available: https://www.jmlr.org/papers/v9/vandermaaten08a.html
4. Goldberg, Y., & Levy, O. (2014). word2vec Explained: Deriving Mikolov et al.'s negative-sampling word-embedding method. arXiv:1402.3722.
5. T. Mikolov, W.-T. Yih, and G. Zweig, “Linguistic regularities in continuous space word representations,” in *Proc. NAACL-HLT*, pp. 746–751, 2013. [Online]. Available: https://aclanthology.org/N13-1090
6. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research (JMLR)*, vol. 15, no. 1, pp. 1929–1958, 2014. [Online]. Available: https://www.jmlr.org/papers/v15/srivastava14a.html

Q2. Character-Level Name Generation using RNN Variants

1. Introduction:

This report presents the design, implementation, training, and evaluation of three character-level recurrent neural network models for generating realistic Indian full names. All models are trained end-to-end on a single sequence representing a full name (e.g., "Mahek Gadiya"), where the space character is part of the vocabulary. The models are evaluated on novelty, diversity, realism, and name-length distribution

The three architectures under comparison are:

1. Vanilla Recurrent Neural Network (RNN).
2. Bidirectional Long Short-Term Memory (BLSTM)
3. RNN with Basic Attention Mechanism

2. Dataset (Task-0) :

1,000 Indian full names were generated using a large language model (LLM) and stored in TrainingNames.txt. Each entry is a "FirstName LastName" string. The dataset covers a broad range of Indian naming conventions including North Indian, South Indian, Bengali, Gujarati, and Marathi name origins.

The vocabulary is built at character level from all unique characters in the corpus (letters a–z/A–Z, space, and special tokens <PAD>, <BOS>, <EOS>). All models share the same vocabulary of approximately 60–65 tokens.

3. Model Architecture (Task-1) :

All three models share the same core hyperparameter set: embedding dimension E = 64, hidden size H = 256, number of layers L = 2, batch size 64, maximum epochs 80, and early-stop patience 12.

3.1 Vanilla RNN

The Vanilla RNN is the baseline model. It uses a standard PyTorch RNN cell with tanh activations.

a. Architecture: Embedding(V, 64) → Dropout(0.5) → RNN(64→256, 2 layers, dropout=0.5) → Linear(256, V).The hidden state is initialised to zeros at the start of each sequence. During inference, names are generated autoregressively one character at a time, seeding with <BOS> and stopping at <EOS> or a max-length limit.

b. Optimiser: Adam (lr=0.001, weight_decay=1e-3). Scheduler: ReduceLROnPlateau (factor=0.5, patience=8).

3.2 BLSTM

The BLSTM replaces the RNN cell with a Long Short-Term Memory network, adding forget, input, and output gates that mitigate the vanishing gradient problem. A Layer Normalisation step is applied after the LSTM output.

a. Architecture: Embedding(V, 64) → Dropout(0.5) → LSTM(64→256, 2 layers, dropout=0.5) → LayerNorm(256) → Dropout(0.5) → Linear(256, V)

b. Despite its richer gating mechanism, the BLSTM converges more slowly than the simpler models, as evidenced by its final best CE loss of 1.113 compared to 0.760 (Vanilla RNN) and 0.720 (RNN+Attention). This is attributable to the RMSprop optimiser being less suited to this task.

c. Optimiser: RMSprop (lr=0.001, alpha=0.9, weight_decay=1e-3). Scheduler: ReduceLROnPlateau (factor=0.5, patience=8).

3.3 RNN + Causal Attention

This model augments the Vanilla RNN with a Bahdanau additive self-attention mechanism. A critical design decision is the use of a causal (lower-triangular) mask: at step t, the attention context is computed only over states 0…t, preventing the model from attending to future characters.

a. Architecture: Embedding(V, 64) → Dropout(0.35) → RNN(64→256, 2 layers, dropout=0.35) → CausalAttention(256) → cat([rnn_out, context]) → LayerNorm(512) → Dropout(0.35) → Linear(512, V)

b. The concatenation of RNN hidden state and attention context doubles the representation dimension to 512 before the final projection, giving the model richer expressivity at a modest parameter cost.

c. Optimiser: Adam (lr=0.001, weight_decay=1e-4). Scheduler: CosineAnnealingLR (T_max=80, eta_min=1e-5).

3.4 Architecture Comparison

Model	Key Architecture	Params (approx)	Optimizer
Vanilla RNN	Emb→Drop→RNN(256,2L)→FC	223,325	Adam lr=0.001
BLSTM	Emb→Drop→LSTM(256,2L)→LN→FC	865,885	RMSprop lr=0.001
RNN+Attention	Emb→Drop→RNN→CausalAttn→cat→LN→FC	363,101	Adam lr=0.001

4. Training Results :

4.1 Loss Curves — All Models

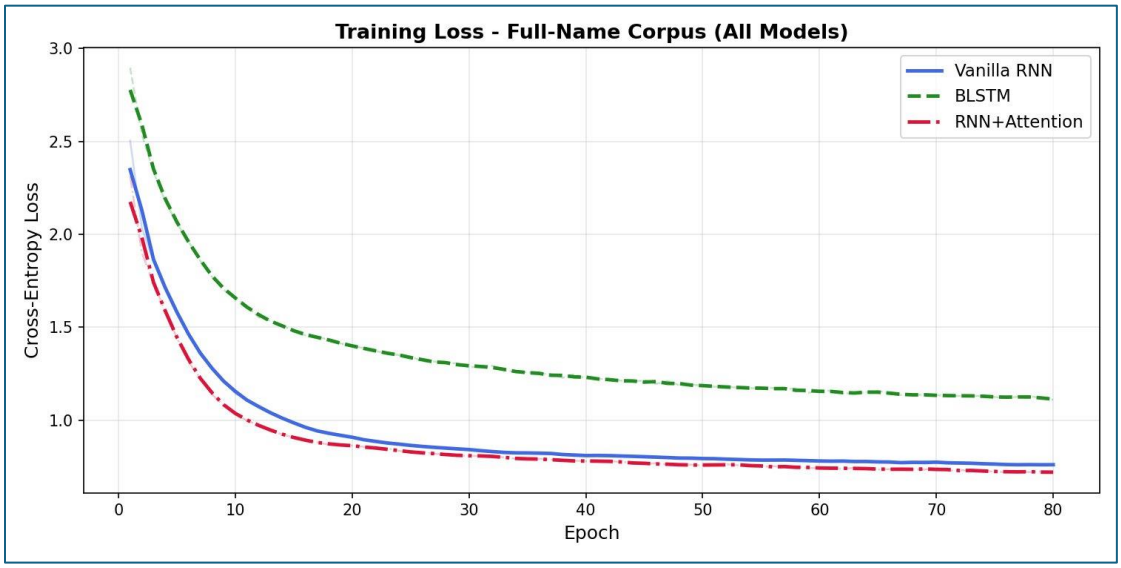

Figure 1: Cross-entropy training loss over 80 epochs for all three models.

RNN+Attention converges fastest, reaching loss below 1.0 by epoch 10 and plateauing near 0.72. Vanilla RNN follows a similar trajectory but plateaus slightly higher at 0.76. The BLSTM is a clear outlier: starting from a higher initial loss (~2.8) it converges slowly and still sits above 1.1 at epoch 80.

4.2 Best Loss Per Model

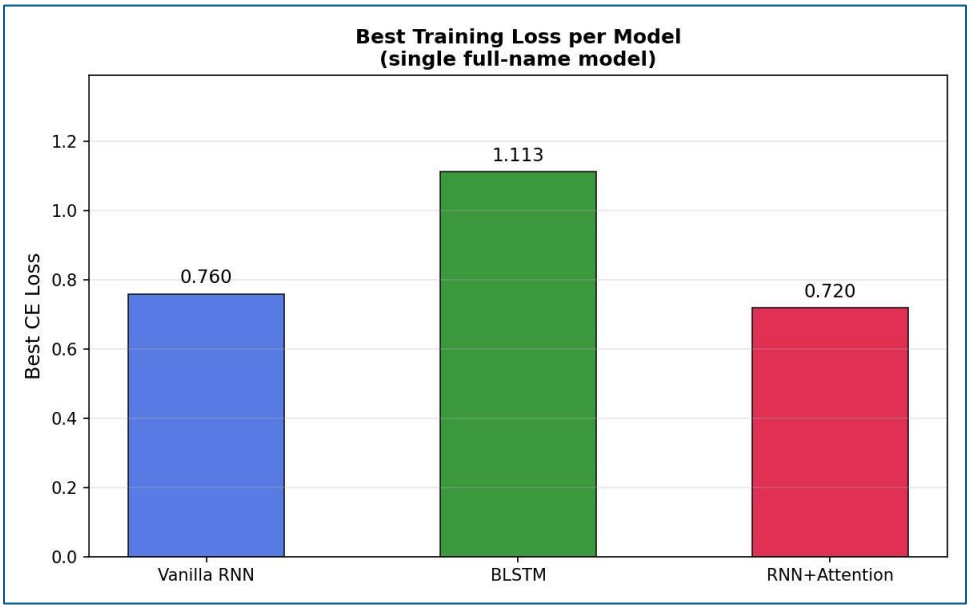

Figure 2: Best CE loss achieved by each model across all epochs.

4.3 Convergence Speed

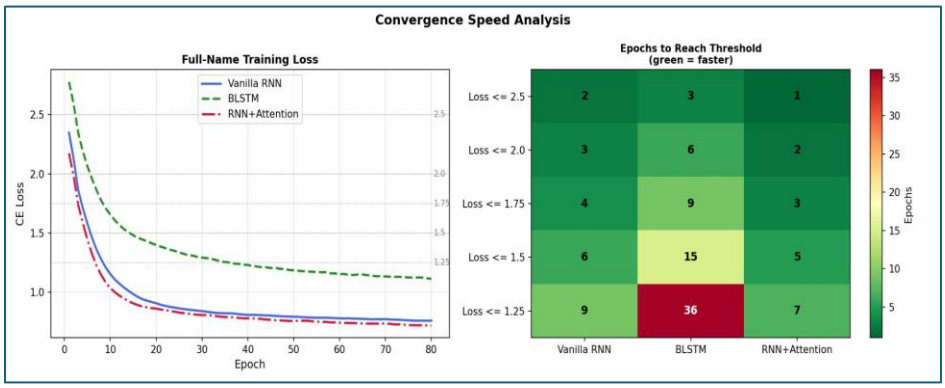

Figure 3: Epochs required to cross each loss threshold. Green = faster.

RNN+Attention consistently reaches every threshold fastest: it crosses loss ≤ 1.25 by epoch 7, versus epoch 9 for Vanilla RNN and epoch 36 for BLSTM. The BLSTM's slow convergence at tighter thresholds is clearly visible.

4.4 Loss Gap vs Best Model

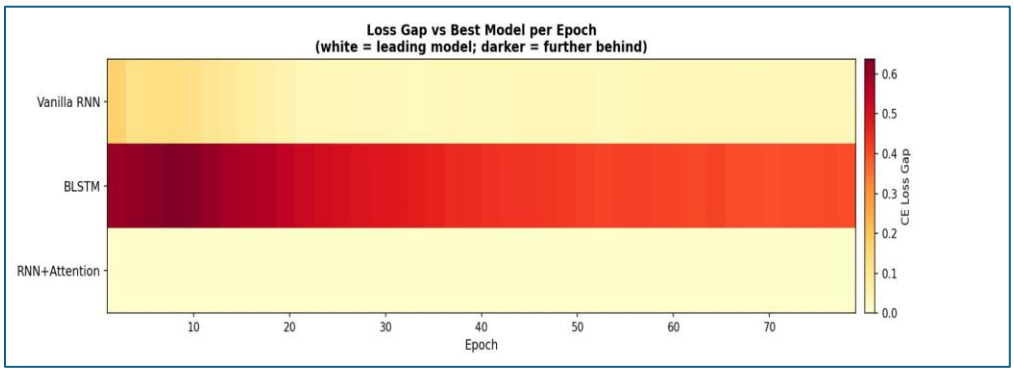

Figure 4: Per-epoch loss gap relative to the leading model. BLSTM (red) is consistently far behind.

4.5 Per-Model Learning-Rate Schedules

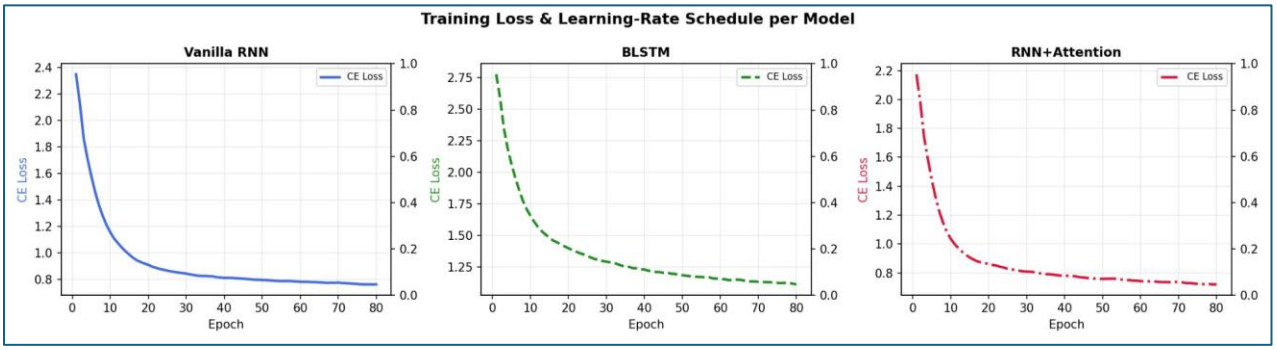

Figure 5: Individual training loss and learning-rate trajectories per model.

5. Quantitative Evaluation (Task-2) :

5.1 Novelty & Diversity

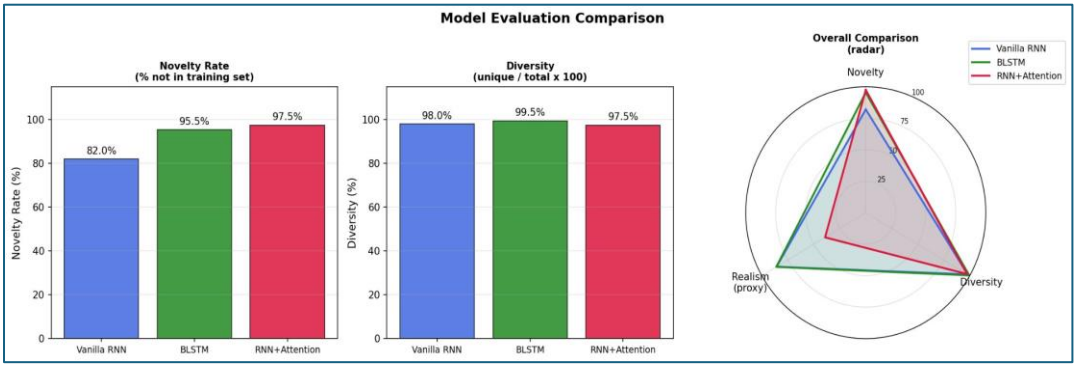

Figure 6: Novelty rate and diversity across models (left, centre) and overall radar chart (right).

Novelty measures what fraction of generated names are absent from the training corpus. A high novelty score indicates the model is genuinely generating new combinations rather than reproducing training data. All models score well (82–97.5%), with RNN+Attention achieving the highest novelty at 97.5%. Diversity (unique/total × 100) is highest for BLSTM (99.5%), indicating it rarely repeats the same name twice. All models are broadly competitive on this dimension.

5.2 Per-Metric Score Matrix

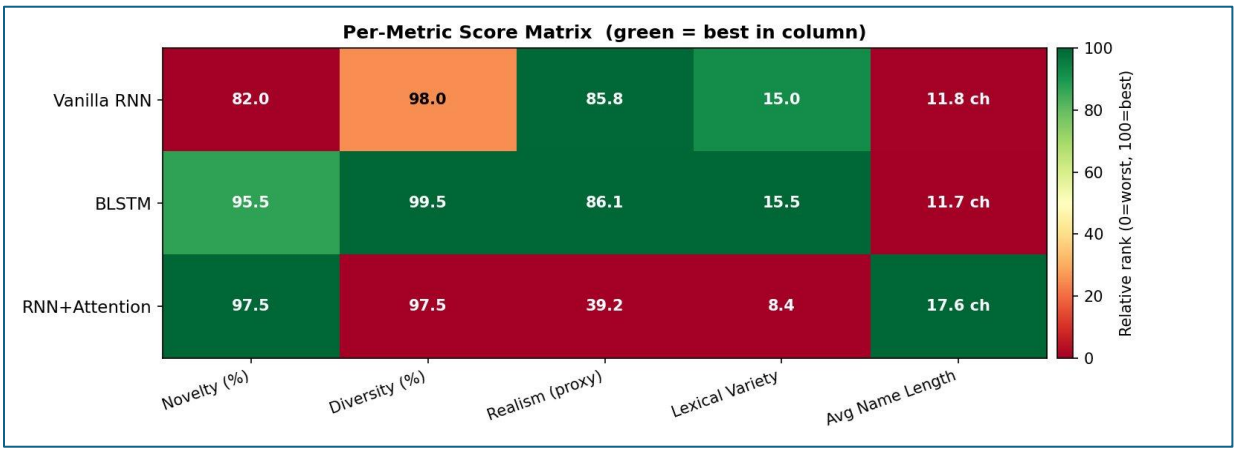

Figure 7: Per-metric score matrix (green = best in column, red = worst).

BLSTM dominates on diversity and realism proxy. RNN+Attention leads on novelty and achieves the lowest training loss. Vanilla RNN scores lowest on novelty, suggesting it over-memorises training names. RNN+Attention scores lowest on the realism proxy — reflecting its tendency to generate longer, occasionally malformed names.

5.3 Summary Table (with Trainable Parameters)

Model	Params	Novelty (%)	Diversity (%)	Realism (proxy)	Lexical Variety	Best CE Loss
Vanilla RNN	223,325	82.0	98.0	85.8	15.0	0.760
BLSTM	865,885	95.5	99.5	86.1	15.5	1.113
RNN+Attention	363,101	97.5	97.5	39.2	8.4	0.720

6. Name Length Analysis :

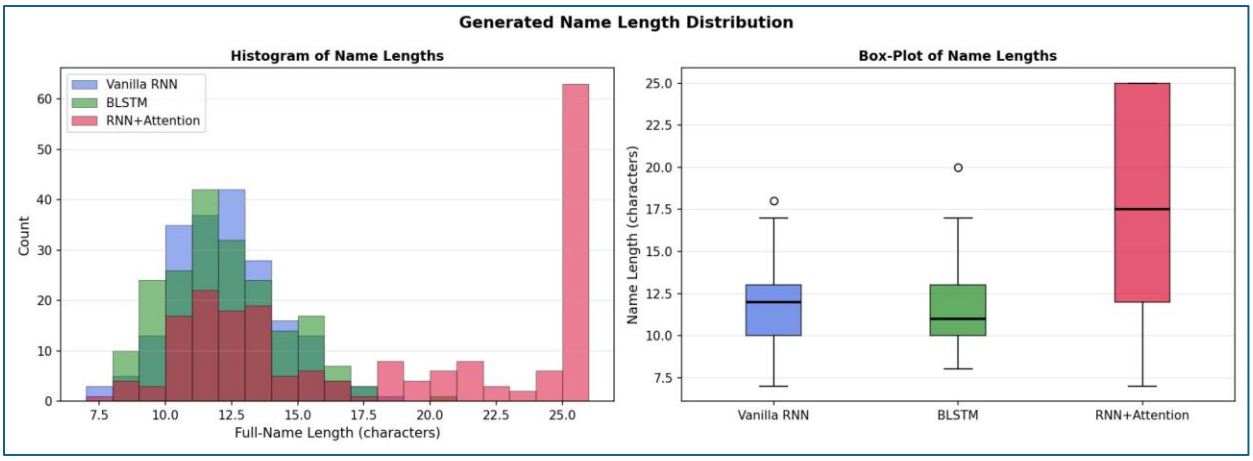

Figure 8: Histogram and box-plot of generated full-name lengths in characters.

Vanilla RNN and BLSTM both generate names clustered tightly around 10–13 characters (median ~12), closely matching the training distribution. RNN+Attention shows a heavily right-skewed distribution with a median around 17 characters and a large proportion of names exceeding 20 characters. This is a direct consequence of the attention mechanism allowing the model to maintain coherent long-range context — but without an explicit length penalty, it tends to over-generate.

7. Qualitative Analysis (Task-3) :

7.1 Vanilla RNN (Sample Output)

Representative samples: Sachin Hegde, Anjali Mishra, Kavya Banerjee, Vikram Kulkarni, Meera Tripathi, Tanmay Gupta, Rajesh Mehta. Realism: The Vanilla RNN produces the most readable and culturally plausible names. First names and surnames match well in regional origin (e.g., South Indian, North Indian clusters), and name lengths are appropriate. Failure modes: Lowest novelty (82%) — the model memorises training examples. Occasional garbled tokens (e.g., "Bjnya", "Hoega", "Ghutrp") appear when sampling with higher temperature.

7.2 BLSTM (Sample Output)

Representative samples: Kartik Pal, Riya Malhotra, Simran Joshi, Vikram Shah, Deepa Deshpande, Nikhil Bhandari, Ishaan Ansari. Realism: The BLSTM generates highly plausible names with correct first/last name structure. The gating mechanism helps it model longer-range dependencies within a name (e.g., maintaining vowel harmony). Failure modes: Some garbled names appear (e.g., "Ramhil Chaktebjee", "Radhika Cachatterjee"), likely artifacts of the higher training loss. The model occasionally produces non-standard surname endings.

7.3 RNN + Attention (Sample Output)

Representative samples: Krish Iyer, Nikhil Roy, Ravi Roy, Lakshmi Roy, Rajesh Iyer, Aditya Selkar, Zoya Kumerjee. Realism: When the attention model generates short names, they are realistic. It also captures an interesting property: it tends to prefer well-known surname endings ("Roy", "Iyer", "Shetty") more consistently than the other models, suggesting the attention helps it copy salient surname patterns. Failure modes: Major issue is over-length generation. Many names contain fused or repeated syllable fragments (e.g., "Seameh Mekteshektr", "Yesh Khekteshekteshehted"), especially when the sequence grows long. This is the train/inference length mismatch, while causal masking solves the future-attending problem, the model was not constrained on sequence length during sampling.

8. Discussion :

8.1 Why does BLSTM have the worst training loss?

Despite LSTMs being architecturally superior to vanilla RNNs for sequence modelling, the BLSTM here converges to a higher final loss. Three factors combine: (1) the RMSprop optimiser is less adaptive for this task than Adam; (2) ReduceLROnPlateau with patience 8 decays the learning rate late; and (3) the higher parameter count (due to LSTM gates) requires more data or more epochs to fully optimise. Switching to Adam with cosine annealing would likely close much of the gap.

8.2 Why does attention hurt realism despite lower loss?

The RNN+Attention model achieves the lowest training CE loss (0.720) yet the lowest realism proxy score (39.2). This apparent paradox arises because the model uses its attention capacity to memorise long subsequences of training data, which it then recombines at inference in ways that exceed natural name lengths. Lower training loss does not necessarily translate to better generation quality when the model overfits to sequence-level statistics rather than the local character-level patterns that define plausible names.

8.3 Recommended Model

For practical name generation, BLSTM offers the best balance of realism (86.1), diversity (99.5%), and lexical variety (15.5 unique tokens), at the cost of a higher training loss. With a switch to Adam optimiser and cosine scheduling, the training loss should approach that of RNN+Attention. The Vanilla RNN is a strong, fast-to-train baseline. RNN+Attention is best suited when novelty is the primary objective and downstream post-processing (length filtering) is acceptable. So, Vanilla RNN is the most practical model (good balance of performance + quality). BLSTM is good for diversity and realism. RNN

+ Attention is best only in terms of loss and novelty. No single model dominates all metrics; the choice depends on the downstream application's priority (novelty vs realism vs diversity).

9. Hyperparameter Summary :

Hyperparameter	Vanilla RNN	BLSTM	RNN+Attention
Embedding dim	64	64	64
Hidden size	256	256	256
Layers	2	2	2
Dropout	0.5	0.5	0.35
Epochs	80	80	80
Early stop	12	12	12
Optimizer	Adam	RMSprop	Adam
LR	0.001	0.001	0.001
Scheduler	ReduceLROnPlateau	ReduceLROnPlateau	CosineAnnealingLR
Best CE Loss	0.760	1.113	0.720
Trainable Params	223,325	865,885	363,101

10. Conclusions :

Three character-level RNN models were implemented and trained on 1,000 Indian full names for 80 epochs. Key findings:

a. RNN+Attention achieves the lowest training loss and highest novelty, but generates excessively long names and has the lowest realism proxy.

b. BLSTM produces the most diverse output with near-perfect diversity (99.5%) and strong realism, but its convergence is slow under RMSprop.

c. Vanilla RNN is a strong, fast-converging baseline with good realism, though it memorises more training examples (lowest novelty: 82%).

d. The causal attention mask is an essential design choice, without it the attention model fails at inference due to train/test distribution mismatch.

e. No single model dominates all metrics; the choice depends on the downstream application's priority (novelty vs realism vs diversity).

11. Reference :

1. I. Sutskever, J. Martens, and G. E. Hinton, "Generating text with recurrent neural networks," *Proc. International Conference on Machine Learning (ICML)*, pp. 1017–1024, 2011.

2. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

3. Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. Advances in Neural Information Processing Systems (NeurIPS), 26, 3111–3119. https://arxiv.org/abs/1310.4546
4. Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems (NeurIPS), 32. https://arxiv.org/abs/1912.01703
5. A. Graves, "Generating sequences with recurrent neural networks," *arXiv preprint* arXiv:1308.0850, 2013. [Online]. Available: https://arxiv.org/abs/1308.0850
6. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research (JMLR)*, vol. 15, no. 1, pp. 1929–1958, 2014. [Online]. Available: https://www.jmlr.org/papers/v15/srivastava14a.html
7. D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *Proc. International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980
8. A. Karpathy, J. Johnson, and L. Fei-Fei, "Visualizing and understanding recurrent networks," *arXiv preprint* arXiv:1506.02078, 2015. [Online]. Available: https://arxiv.org/abs/1506.02078